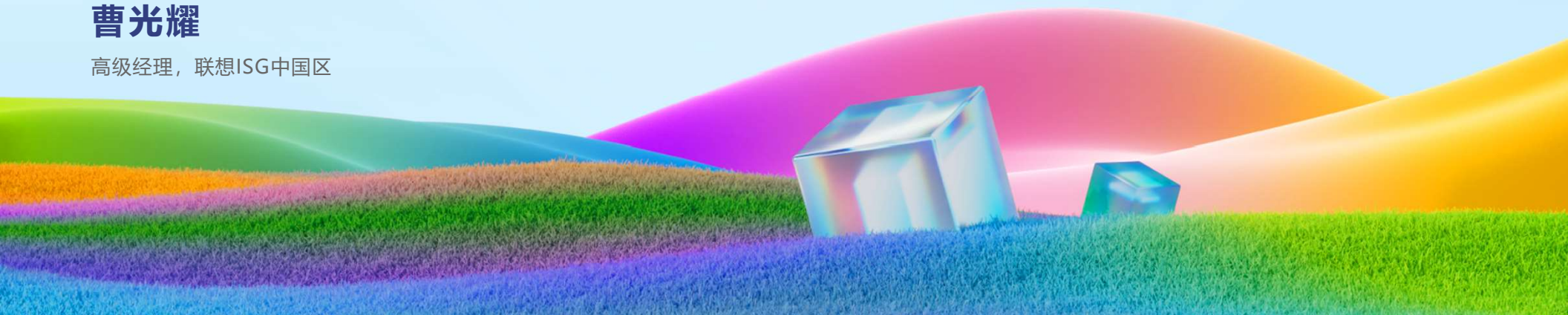


云固件的实践和展望

曹光耀

高级经理，联想ISG中国区



目录

01 云固件简介

02 云固件的现状

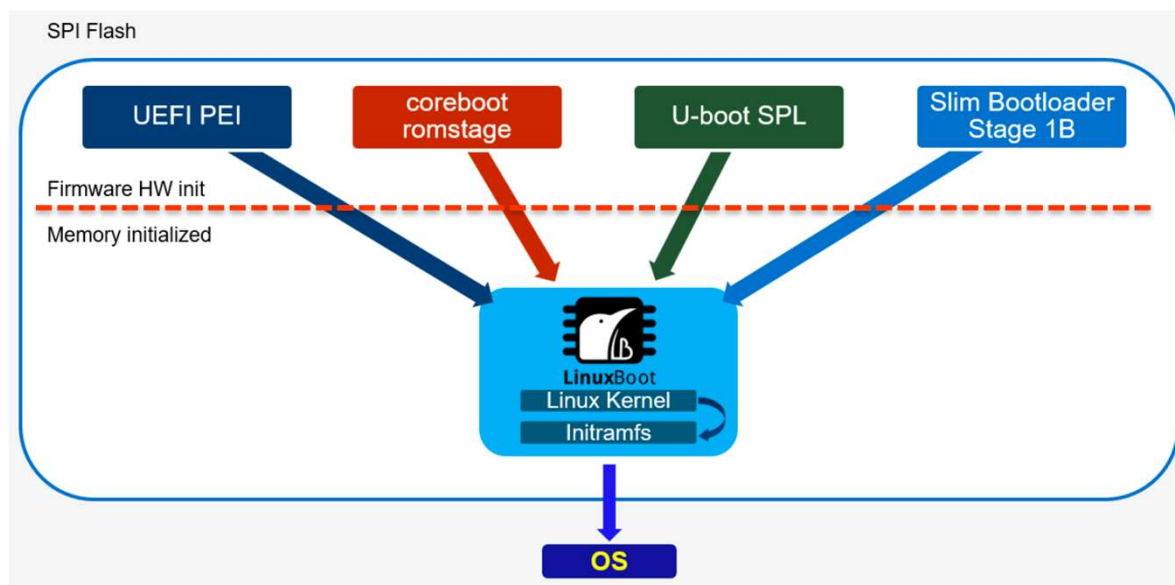
03 云固件的关键问题

04 云固件的发展展望

云固件简介

为什么是LinuxBoot with u-root

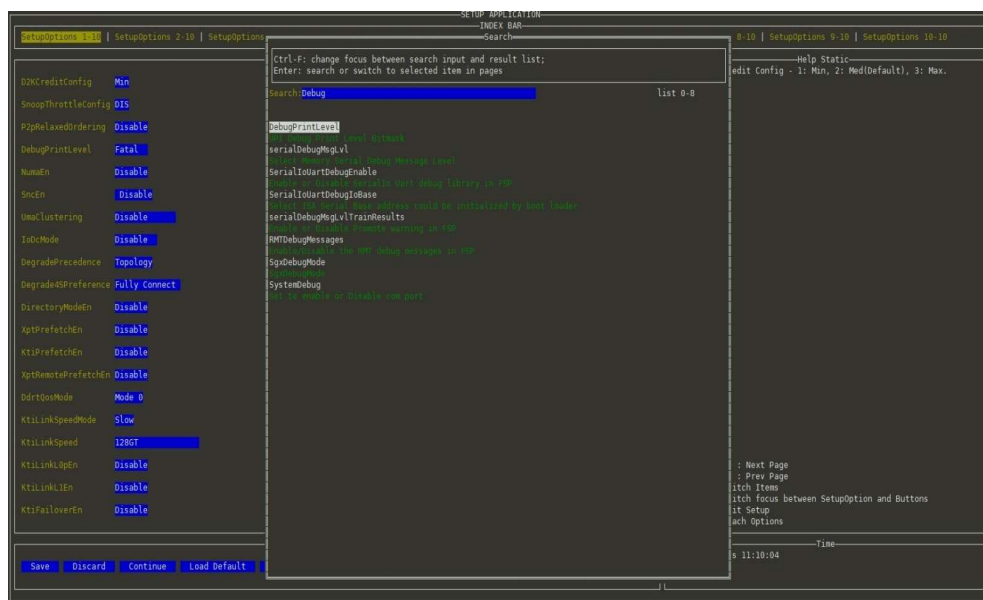
- 设备驱动：沿用Linux kernel驱动，驱动健壮性经过Linux各个发行版检验，更加成熟稳定。
- 强大的功能开发环境：在LinuxBoot成熟多线程和golang语言加持下，BIOS功能开发效率得到大幅提升。
- 活跃的社区环境支持：无论是linux还是golang开源社区活跃程度都较高。



云固件的现状

基于LinuxBoot的开发进展-功能补齐

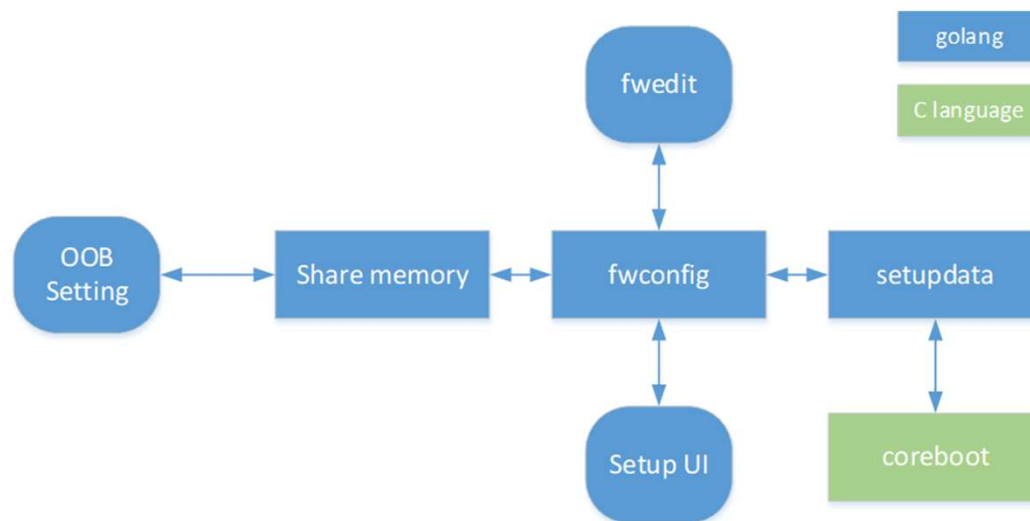
- Setup界面: 提供基于golang开发的轻量级setup UI。自动抓取所有系统选项配置信息自动生成。支持分页切换, 保存, 恢复默认值和搜索等功能
- 变量编辑工具fwedit: 支持选项整体/部分导入导出, 默认值/当前值修改功能以及密码等相关安全功能
- Redfish: 支持资产信息收集和发送, 基于redfish的选项值修改
- 其他UEFI BIOS支持的功能



云固件的现状

基于LinuxBoot的开发进展-配置数据分发

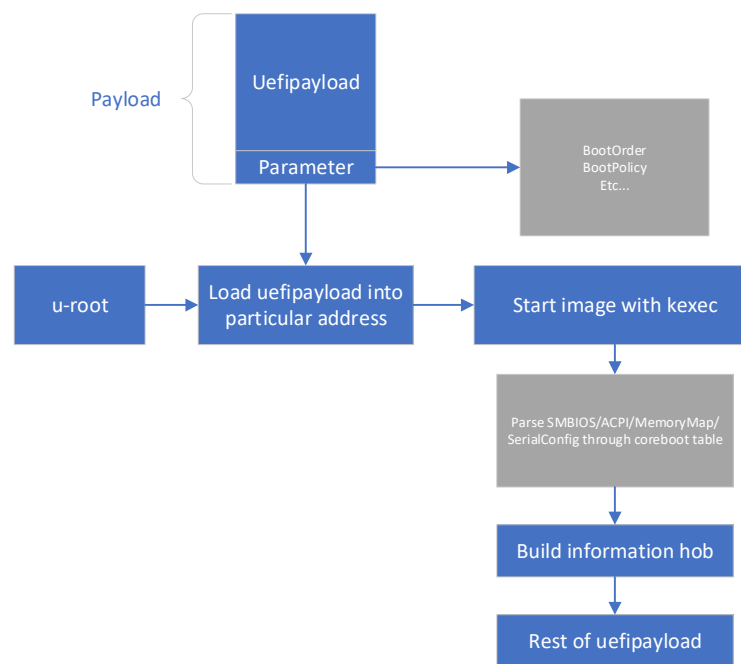
- 以fwconfig为核心定义数据交换格式，隔离底层架构和平台相关代码。
- 所有修改方法最终汇总到一个数据通路，避免冲突和不一致
- 蓝色部分跨平台可直接复用，绿色平台相关代码需要适配底层分发架构和数据格式



云固件的现状

EFI系统的安装和启动-with UefiPayload

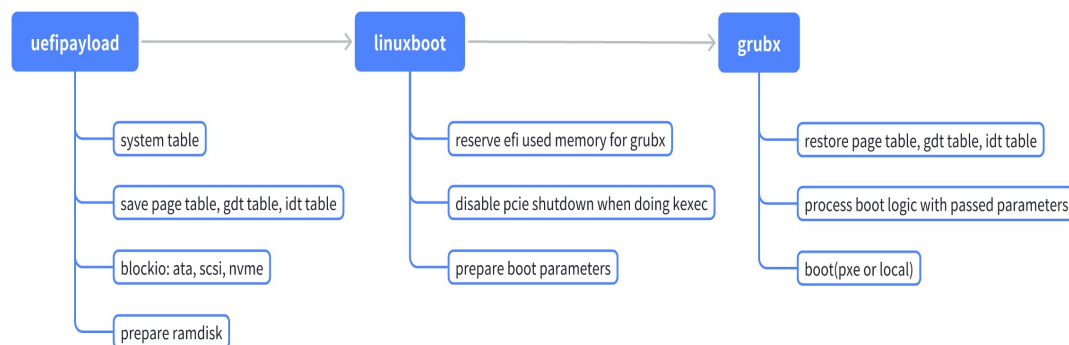
- 跳转: 在u-root界面解析 UefiPayload , 通过kexec完成跳转。
- 构建: 在UefiPayload中利用 coreboot table重新构建 SMBIOS/ACPI/MemoryMap等信息, 构建最小的BootService和 RuntimeService填入 SystemTable中。
- 启动: 通过UEFI Driver安装或者启动相应设备。



云固件的现状

EFI系统的安装和启动-with Grubx

- UefiPayload阶段主要负责UEFI环境创建，并在内存中驻留，备后续使用
- LinuxBoot阶段主要负责接收用户输入，并针对用户指令做出响应的动作，包括启动策略制定
- Grubx阶段主要负责恢复UEFI执行环境，引导UEFI环境操作系统

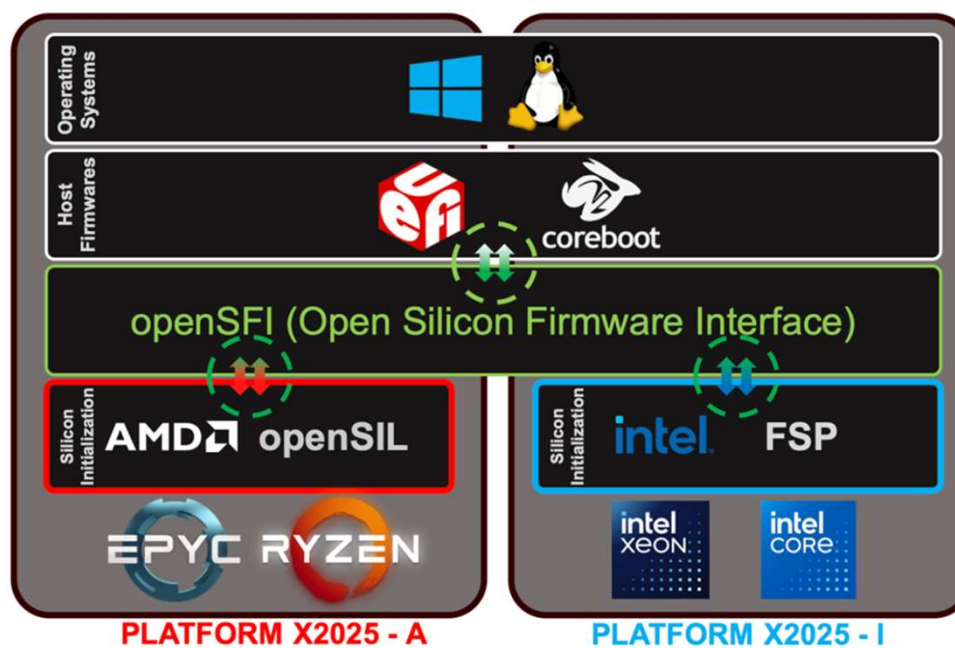


云固件的关键问题

- 如何在LinuxBoot方案作为BIOS情况下，支持当前UEFI compatible的系统？
- 如果只需要支持Linux系统，是否仍要保留Grub支持？

云固件的发展展望

- 芯片初始化接口更加统一
- 系统固件产品化，平台代际延续性更强
- 基于开源代码维护一套自研 codebase 将更加简单



Call for action

云固件子项目wiki:

https://www.opencompute.org/w/index.php?title=OCP_Regional_Community:_PRC/Cloud-Firmware

联系人:

shuo.liu@ocproject.net

keno.li@ocproject.net

guangyao.cao@ocproject.net